Semana 1. Microcontroladores, bits y registros

1. Del computador al microcontrolador

Un **sistema embebido** utiliza un microcontrolador para realizar una tarea concreta, como controlar una alarma, una lavadora o un termostato. Un **microprocesador** se concentra en la CPU y utiliza memorias y periféricos externos; un **microcontrolador** integra CPU, memorias y entradas/salidas en un chip, con bajo consumo.

Dentro de la CPU están la **ALU** (operaciones y lógica), los **registros** (datos rápidos), la **unidad de control** y el **contador de programa (PC)**. El trabajo se repite en tres pasos: buscar, decodificar y ejecutar instrucciones.

Bus	Función
Direcciones	Indica qué posición de memoria o registro se utiliza.
Datos	Transporta el valor que se lee o escribe.
Control	Señala si la operación es lectura o escritura.

2. Arquitectura y memorias

En **von Neumann**, programa y datos comparten el acceso a memoria. En **Harvard** se utilizan caminos separados, lo que permite acceder a instrucciones y datos de forma simultánea. El ATmega328P trabaja con arquitectura Harvard.

Memoria	Uso	¿Conserva datos sin energía?
Flash	Guarda el programa.	Sí
SRAM	Guarda variables durante la ejecución.	No
EEPROM	Guarda configuraciones pequeñas.	Sí

El **ATmega328P** del Arduino Uno es de 8 bits, 16 MHz y trabaja a 5 V. El **ESP32** es de 32 bits, puede usar dos núcleos, tiene Wi-Fi/Bluetooth y sus pines trabajan a 3,3 V.

3. Ejemplo de acceso a memoria

Para escribir **0010 0000** en la dirección **0x25**, la CPU coloca 0x25 en el bus de direcciones, el byte en el bus de datos y activa la señal de escritura. Para leer se hace el proceso inverso, con la señal de lectura.

4. Bits, binario y hexadecimal

Un **bit** vale 0 o 1; un **byte** tiene 8 bits con pesos 128, 64, 32, 16, 8, 4, 2 y 1. Para pasar a decimal se suman los pesos de los bits que valen 1. Un dígito hexadecimal representa 4 bits y se escribe con el prefijo **0x**.

Ejercicios: 1000 0001 = 128 + 1 = **129** = **0x81**. 0xFF = 1111 1111 = **255**. 200 = 128 + 64 + 8 = **1100 1000** = **0xC8**.

Otro ejemplo: 0x20 | 0x01 une los bits activados de ambos valores: 0010 0000 | 0000 0001 = **0010 0001** = **33**.

5. Operaciones bit a bit y máscaras

Operación	Símbolo	Resultado
AND	&	Da 1 si ambos bits son 1.
OR	|	Da 1 si al menos uno es 1.
XOR	^	Da 1 si son distintos.
NOT	~	Invierte 0 y 1.

Una **máscara** como **1 << n** selecciona el bit n sin modificar los demás. Con un registro REG:

```
REG |=  (1 << n);   // encender bit n
REG &= ~(1 << n);   // apagar bit n
REG ^=  (1 << n);   // invertir bit n
REG &   (1 << n);   // consultar bit n
```

Ejercicio: si REG = 1010 1010, entonces REG |= (1 << 0) da **1010 1011**; REG &= ~(1 << 7) da **0010 1010**; REG ^= 0x0F da **1010 0101**. Cada operación parte del valor inicial.

6. Entradas y salidas en AVR

DDRx decide entrada (0) o salida (1); **PORTx** escribe HIGH/LOW o activa la resistencia pull-up cuando el pin es entrada; **PINx** permite leer el nivel real del pin.

```
DDRB  |=  (1 << PB5);   // PB5 salida (LED)
PORTB |=  (1 << PB5);   // encender LED
DDRD  &= ~(1 << PD2);   // PD2 entrada
PORTD |=  (1 << PD2);   // activar pull-up
bool pulsado = !(PIND & (1 << PD2));
```

Resultado: con un botón conectado a GND y pull-up activada, el pin lee **HIGH** al estar suelto y **LOW** cuando se presiona.